

Conductor Labs MVP

1. Product Vision

Conductor Labs is the definitive single source of truth for release readiness. It eliminates deployment anxiety by giving engineering teams absolute, real-time confidence in whether a release is safe to ship.

2. Problem Statement

Before a deployment, engineering teams suffer from "tab fatigue." To answer "*Are we ready?*", they must manually cross-reference GitHub PRs, Linear issues, CI/CD pipelines, and code reviews. This fragmentation creates blind spots, slows down deployment velocity, and leads to broken releases being shipped because a single unchecked dependency fell through the cracks.

3. User Personas

For the MVP, we are primarily focusing on Engineering Managers, Lead Managers and Startup Founders.

- **The Shipper (Engineering Manager / Lead Dev):** The person who actually presses "deploy." They need a high-level aggregate view of readiness.
- **The Contributor (IC Developer):** The person whose code is in the release. They need to know exactly which of their PRs or CI runs are blocking the release.

4. User Stories

- **As a Shipper**, I want to see a single "Ready/Not Ready" status for an upcoming release, so I can deploy with confidence without checking 4 different apps.
- **As a Shipper**, I want to see exactly which checks (e.g., failed CI, unreviewed PR) are blocking readiness, so I can unblock them.
- **As a Contributor**, I want to see a unified, chronological timeline of all events (commits, PRs, issue closures, CI runs) attached to a release, so I understand the state of the codebase at a glance.
- **As an Engineering Manager**, I want to know exactly what is blocking a release without opening GitHub or Linear.

5. MVP Scope

The MVP will do exactly two things flawlessly. Nothing else.

1. **The Unified Timeline:** A chronological ledger of everything tied to a release. GitHub events, Linear issue updates, and CI statuses interweaved into a single, latency-free feed.
2. **The Readiness Gate:** A deterministic evaluation engine. A release is either Ready or

Blocked. The engine checks specific, non-negotiable rules (All tied PRs merged + All tied CI passing + All tied Linear issues closed).

6. Out of Scope (The "No" List)

- **AI of any kind:** We deal in deterministic facts, not probabilistic guesses.
- **Analytics & DORA metrics:** We are an operational tool, not a reporting tool.
- **Notifications (Slack/Email):** Users come to Conductor for truth, we don't spam them.
- **Project Management:** No creating, editing, or dragging issues. Read-only.
- **Deployment Execution:** We do not ship the code. We only tell you if you *should*.
- **Custom Rule Builders:** MVP rules are hardcoded and opinionated.
- **RBAC (Role-Based Access Control):** Everyone in the workspace sees everything.

7. Functional Requirements

- **OAuth Integrations:** Must authenticate with GitHub and Linear.
- **Webhook Ingestion:** Must consume webhooks from GitHub (PRs, Checks, Reviews) and Linear (Issue status changes) in real-time.
- **Release Definition: A Release is branch-based.** When a user creates a Release they choose one Git branch. Every readiness check and timeline event belongs to that branch.
- **Readiness Calculation:** PR must target the tracked branch. CI must be the latest run for the latest commit. Linear issue must be linked to one of those PRs
- **Chronological Feed:** System must parse incoming webhooks, normalize them into standard "Events," and display them sorted by timestamp.

8. Non-functional Requirements

- **Near Real-Time UI:** The dashboard must update instantly via WebSockets/Server-Sent Events when a webhook is received. A dev shouldn't have to refresh to see CI turn green.
- **Zero UI Clutter:** The interface must be stark, fast, and keyboard-navigable (think Linear).
- **Idempotency:** Webhook ingestion must handle duplicate deliveries gracefully without creating duplicate timeline events.

9. User Flow: Creating & Checking a Release

1. Workspace Setup

User Action

- User signs up and creates a Workspace.
- User authenticates GitHub via OAuth.

- User authenticates Linear via OAuth.

System Action

- Store OAuth credentials securely.
 - Verify access to the selected GitHub repositories.
 - Begin listening for incoming webhooks from GitHub and Linear.
-

2. Create a Release

User Action

- Click **Create Release**.
- Enter a release name (e.g. **July Release**).
- Select a GitHub repository.
- Select the branch to track (e.g. **main** or **release-v2**).

System Action

- Create a Release record.
 - Associate the Release with the selected repository and branch.
 - Begin the initial repository scan.
-

3. Initial Repository Scan

System Action

Conductor retrieves the current state of the selected branch and builds the initial release state.

It discovers:

- Open and merged Pull Requests
- Latest commits
- CI/CD workflow results
- Linked Linear issues
- Existing review status

The initial Readiness status is calculated immediately.

4. Real-Time Synchronization

After the initial scan, Conductor switches to real-time updates.

Whenever GitHub or Linear sends a webhook:

- Pull Request opened
- Pull Request merged
- Review submitted
- CI completed
- Commit pushed
- Linear issue updated

Conductor:

- Normalizes the event.
- Stores it as a Timeline Event.
- Recalculates Release Readiness.
- Pushes updates to connected clients via WebSockets.

The dashboard updates automatically without requiring a page refresh.

5. Release Detail View

When a user opens a Release they immediately see:

Readiness Summary

A single deployment status:

-  Ready
-  Blocked

If blocked, every blocking item is listed.

Example:

- PR #204 awaiting review
 - GitHub Actions failing
 - Linear Issue ENG-142 still In Progress
-

Timeline

A chronological timeline showing every event that affects the Release.

Examples:

- Commit pushed
- Pull Request opened
- Review approved
- CI passed
- Linear issue moved to Done

This gives engineers a single, unified history of everything that happened before deployment.

6. Release Ready

When every readiness rule passes:

- All required Pull Requests are merged.
- All required CI checks pass.
- All linked Linear issues are completed.

The Release status automatically changes to **Ready**.

At this point Conductor's responsibility ends.

The product **does not deploy code**.

It simply gives the engineering team confidence that the release is ready to ship.

10. Wireframe Descriptions

- **Screen 1: Global Workspace (Releases Index)**
 - A simple, clean list of all active Releases.
 - Columns: Release Name, Branch, Last Activity, and a stark status badge:  Ready or  Blocked.
- **Screen 2: Release Detail (The Core View)**
 - **Header:** Release Name and Branch.
 - **Top Section (Readiness):** A massive visual indicator. If blocked, a list immediately below shows exactly why (e.g.,  PR #402 missing review,  CI: GitHub Actions failed).
 - **Bottom Section (Timeline):** A vertical, chronological feed. Minimalist icons (GitHub logo, Linear logo, Checkmarks, Crosses) with one-line descriptions (e.g., *"Issue ENG-142 moved to Done by @alex"*).

11. Database Entities (High Level)

Entity	Purpose	Key Attributes
Workspace	Represents an organization or team using Conductor.	id name slug created_at updated_at
Integration	Stores OAuth credentials for external services.	id workspace_id provider (github, linear) access_token refresh_token connected_at
Release	Represents a deployment candidate being tracked.	id workspace_id name repository target_branch status (READY BLOCKED) created_at updated_at
Event	Represents every external event that affects a release timeline.	id release_id source (github, linear) type timestamp

Entity	Purpose	Key Attributes
		raw_payload (JSONB) external_id created_at
ReleaseCheck	Represents one requirement that contributes to Release Readiness.	id release_id check_type external_id status last_updated

12. API Endpoints (High Level)

Method	Endpoint	Purpose
POST	/api/webhooks/github	Ingests PR, commit, and CI status events
POST	/api/webhooks/linear	Ingests issue status changes
GET	/api/releases	Lists releases for the workspace
POST	/api/releases	Creates a new release tracker
GET	/api/releases/:id	Returns Readiness score and current state
GET	/api/releases/:id/timeline	Returns paginated chronological events

13. Future Roadmap (Post-MVP)

Keep in mind, we only build these if they directly enhance the binary decision of "Are we ready?"

1. **Advanced Rules Engine:** Allow teams to define custom readiness logic (e.g., "Ready only if code coverage > 80%").
2. **API/CLI Gate:** Provide a CLI tool (conductor check) so teams can block their actual deployment scripts automatically if Conductor returns Blocked.
3. **More Data Sources:** Sentry (block release if new critical errors detected on staging), Datadog, etc.

14. Open Product Questions (To answer before you write code)

The "Mapping" Problem: How exactly does Conductor know which Linear issues and which GitHub PRs belong to a specific Release?

Does the user manually attach them? Do we parse branch names (e.g., release-v1 branch contains commits with PR numbers)? Do we rely on GitHub's built-in issue linking? If we get this mapping logic wrong, the Readiness score will be inaccurate, and users will immediately abandon the product.